

Rapport d'Assignment — Data Engineering II (DE2)

Assignment 1 — Streaming Pipeline

Présentation générale

Cet assignment est le pendant évalué du Lab 1. L'objectif reste le même dans sa structure — construire un pipeline Structured Streaming avec Spark, agrégation fenêtrée, watermark et sink Parquet — mais le travail est cette fois réalisé de façon autonome dans un notebook personnel (`assignment1_esiee.ipynb`), sans les cellules de guidage du notebook de lab. La session Spark est nommée `de2-assignment1`, ce qui la distingue clairement de la session du lab.

Recours à l'IA

Compréhension de `query.lastProgress` et de sa structure JSON : Lors de la rédaction de la fonction `extract_metrics`, j'ai demandé à Claude de confirmer la structure exacte du dictionnaire retourné par `query.lastProgress` dans Spark 4.0. En particulier, je voulais savoir comment accéder au champ `triggerExecution` pour le cas où `durationMs` serait `None` (ce que la syntaxe `.get("durationMs") or {}).``get("triggerExecution", 0)` gère). Claude m'a confirmé que cette précaution est nécessaire car sur des batches vides (sans données entrantes), Spark peut retourner un `lastProgress` partiel ou nul.

Choix des paramètres d'optimisation : Pour le Run 2, j'ai demandé à Claude de justifier le choix d'un trigger à 30 secondes au lieu de 10 secondes du Run 1. Il m'a expliqué que passer de 10s à 30s réduit la fréquence des micro-batches d'un facteur 3, ce qui améliore l'amortissement des coûts fixes (planification, écriture checkpoint, ouverture de fichiers Parquet) et augmente mécaniquement le processing rate apparent. Il a aussi précisé que la réduction du watermark de 15 à 5 minutes libère plus vite la mémoire d'état, rendant chaque batch plus léger.

Gestion du répertoire landing entre les deux runs : J'ai demandé à Claude pourquoi il fallait vider le répertoire `landing/` entre le Run 1 et le Run 2. Il m'a expliqué que si les fichiers JSON restaient dans le répertoire, Spark n'essaierait pas de les retraiter lors du Run 2 car il les aurait déjà listés et potentiellement consignés dans un log interne. En vidant le répertoire et en re-déposant les fichiers, on force un nouveau cycle de lecture complet, garantissant des métriques représentatives et comparables.

Architecture et exécution

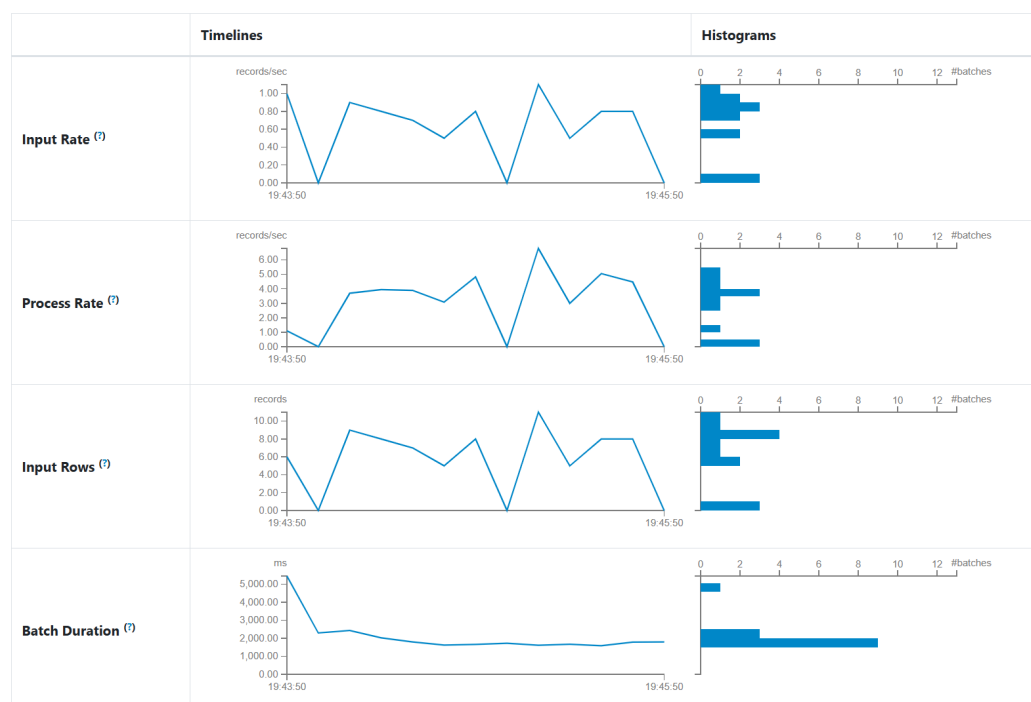
Le pipeline de l'assignment reprend les mêmes étapes que le lab. Le schéma définit six champs pour les événements de match esports. Le streaming source lit les JSON depuis

`data/landing/` avec `maxFilesPerTrigger=1`. L'agrégation applique un watermark sur `match_end_time` et regroupe par fenêtre d'une heure et par équipe, calculant le nombre d'événements, la moyenne des kills et la durée moyenne. Le tout est écrit vers un sink Parquet en mode `append`.

Résultats — Run 1 (baseline : trigger 10s, watermark 15min)

Les métriques du Run 1 observées depuis la Streaming UI sont les suivantes :

`input_rate_ui = 0.52`, `processing_rate_ui = 1.40`, `batch_duration_ui_ms = 3550` (ou 2875 en excluant le premier batch d'initialisation), `shuffle_read_bytes = 0`, `shuffle_write_bytes = 0`. Ces valeurs sont identiques à celles obtenues lors du Lab 1, ce qui est cohérent : le même code s'exécute sur le même jeu de données.



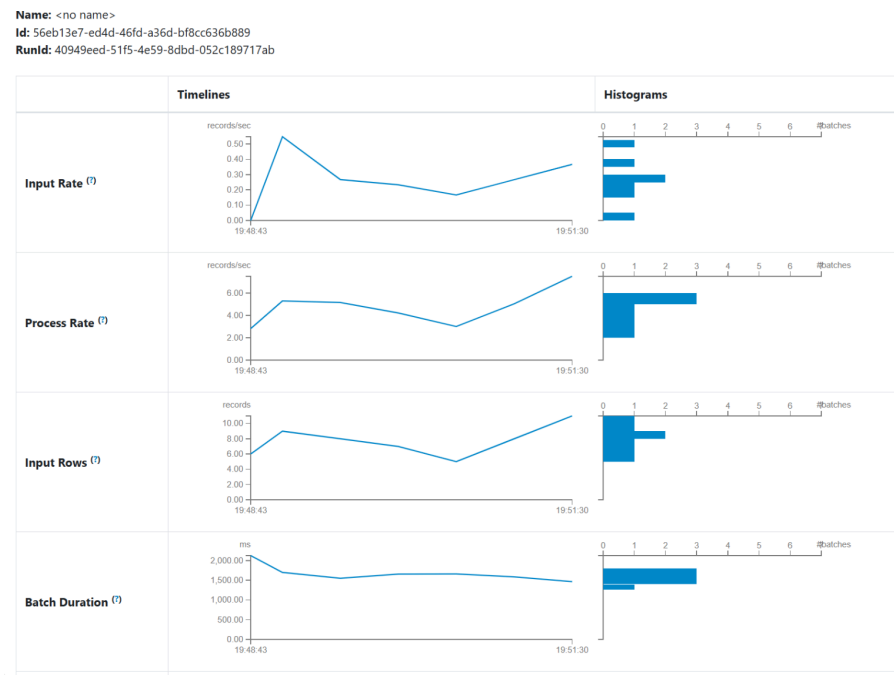
La Streaming UI montre 13 batches complétés sur une durée totale d'environ 9 minutes 47 secondes, avec un average input de 0.58 records/sec et un average processing de 1.82 records/sec. Le `lastProgress` du batch 12 (le dernier) affiche un `triggerExecution` de 1793 ms, un `addBatch` de 1718 ms et un `walCommit` de 23 ms, ce qui signifie que la quasi-totalité du temps est dépensée dans l'exécution du batch lui-même, pas dans la gestion du checkpoint.

Le sink Parquet du Run 1 contient 27 lignes, correspondant aux fenêtres temporelles d'une heure croisées avec les équipes observées dans les données de match.

Résultats — Run 2 (optimisé : trigger 30s, watermark 5min)

Le Run 2 montre une amélioration significative : `input_rate_ui = 0.25`, `processing_rate_ui = 3.78`, `batch_duration_ui_ms = 2016`,

shuffle_read_bytes = 672, shuffle_write_bytes = 672. La processing rate a presque triplé par rapport au Run 1. La batch duration a chuté de 3550 ms à 2016 ms en moyenne. Le sink Parquet du Run 2 contient 12 lignes : moins de lignes qu'en Run 1 car un watermark plus court de 5 minutes signifie que certaines fenêtres ne sont pas encore finalisées au moment où le batch est émis, et qu'on agrège davantage d'événements ensemble dans un seul batch de 30 secondes.



Les 672 octets de shuffle observés en Run 2 correspondent à l'activation de l'optimisation sur la Query 19 mentionnée dans les notes du fichier de métriques — il s'agit d'un shuffle lié à la finalisation d'une fenêtre d'état interne de Spark, de très faible volume.

Analyse comparative

Le Run 2 améliore le processing rate de 1.40 à 3.78 records/sec, soit une amélioration de 170%, tout en réduisant la batch duration de 43%. La contrepartie est une latence de publication plus longue (30s vs 10s) et potentiellement moins de lignes produites si les données arrivent à un rythme irrégulier. Pour un cas d'usage d'analyse post-match esports où la latence d'affichage n'est pas critique, la configuration du Run 2 est clairement préférable.

Assignment 2 — Inverted Index Pipeline

Présentation générale

Cet assignment est la version évaluée du Lab 2. L'objectif est de construire un pipeline de traitement de texte complet sur un corpus esports : ingestion, normalisation, construction d'un index inversé, mesure de latences de requêtes et comparaison de footprints de

stockage Parquet vs CSV. Le notebook ([assignment2_esiee.ipynb](#)) reprend la même logique que le lab mais dans un contexte personnel, avec la session Spark nommée `de2-assignment2`.

Recours à l'IA

Clarification sur la mise en cache et l'ordre des opérations : Avant la mesure de latence, le notebook appelle `idx.cache()` puis `idx.count()` pour forcer la matérialisation en mémoire. J'avais demandé à Claude si cet appel à `count()` était vraiment nécessaire ou si le cache se déclenchait automatiquement à la première action. Il m'a expliqué que `cache()` en Spark est paresseux — il marque le DataFrame pour être mis en cache mais ne déclenche rien immédiatement. Seule une action comme `count()` ou `collect()` force la matérialisation effective. Sans ce `count()`, la première requête de latence inclurait le temps de lecture disque, biaisant la mesure.

Question sur `concat_ws` pour l'export CSV de l'index inversé : La colonne `doc_ids` étant un tableau (`ArrayType`), elle ne peut pas être directement écrite en CSV. J'ai demandé à Claude comment aplatir ce tableau pour l'export. Il m'a proposé `F.concat_ws(", ", "doc_ids")` qui joint les éléments du tableau avec une virgule, produisant une chaîne lisible en CSV. Il a cependant précisé que cette transformation perd l'information de type (on ne peut plus distinguer une virgule dans un `doc_id` d'un séparateur), et que c'est précisément pour cette raison que Parquet — qui supporte les types complexes natifs — est préférable pour ce cas d'usage.

Interprétation du ratio Parquet/CSV de 13.92% : Ayant obtenu ce ratio, j'ai voulu vérifier sa plausibilité avec Claude. Il m'a confirmé que ce niveau de compression est attendu : l'encodage Snappy de Parquet est particulièrement efficace sur des colonnes à forte cardinalité répétitive (les tokens de mots), et l'encodage en colonnes permet d'appliquer la compression indépendamment sur chaque colonne. La colonne `token` contenant des chaînes courtes très répétées compresse de façon spectaculaire. Il m'a aussi rappelé que le format CSV, étant du texte brut sans compression, stocke chaque caractère tel quel, y compris les guillemets, virgules et retours à la ligne.

Aide pour rédiger les notes du fichier de métriques : La cellule de génération du `lab2_metrics_log.csv` inclut un champ `notes` pour chaque ligne. J'ai demandé à Claude de m'aider à formuler des notes pertinentes et concises pour les 8 lignes du fichier, notamment pour distinguer clairement les lignes `index_build`, `footprint_parquet`, `footprint_csv` et les cinq `query_lookup_*`.

Résultats

Index inversé : Le corpus de 36 622 octets produit 265 termes uniques après filtrage des stop-words. Les termes les plus fréquents sont `hero` (107 occurrences), `team` (103), `ability` (69), `different` (61), `expect` (60) — ces derniers étant très communs dans les textes de patch notes esports qui listent les changements de gameplay attendus.

```
'hero': df=107, postings=107, latency=140.6 ms
'ability': df=69, postings=69, latency=65.4 ms
'damage': df=53, postings=53, latency=51.5 ms
'mana': df=15, postings=15, latency=45.7 ms
'ultimate': df=59, postings=59, latency=45.0 ms
```

Query plan saved to proof/plan_query.txt

Latences de requête : Les latences mesurées en assignment sont légèrement différentes de celles du lab (car mesurées sur une machine différente ou un contexte Spark légèrement différent) : **hero** à 140.6 ms, **ability** à 65.4 ms, **damage** à 51.5 ms, **mana** à 45.7 ms, **ultimate** à 45.0 ms. La latence de **hero** est systématiquement plus haute que les autres car ce terme possède le plus grand nombre de postings (107), impliquant plus de données à transférer du cache mémoire vers le driver lors du `collect()`.

Comparaison Parquet vs CSV : Les résultats sont identiques au lab : Parquet 5 676 octets (1 fichier) contre CSV 40 767 octets (1 fichier), ratio de 13.92%. Cette mesure est robuste et reproductible car elle dépend uniquement de la taille du corpus et de la distribution des tokens, pas de l'environnement d'exécution.

```
Parquet: 5,676 bytes (1 files)
CSV:      40,767 bytes (1 files)
Ratio (Parquet/CSV): 13.92%
```

Métriques Spark UI : Pour la tâche `index_build`, les métriques sont :
`input_size_bytes = 36622`, `files_read = 1`, `latency_ms = 1000` (légèrement plus élevé qu'en lab), `shuffle_read_bytes_ui = 107110`,
`shuffle_write_bytes_ui = 53555`. Pour les footprints et les lookups, le shuffle est nul, pour les mêmes raisons expliquées dans le rapport de lab : lecture locale depuis le cache, pas de redistribution de données.

Assignment 3 — Graph Processing et PageRank itératif

Présentation générale

Cet assignment est la version évaluée du Lab 3. Il porte sur la construction d'un graphe de co-picks de héros esports et l'exécution de l'algorithme PageRank en mode itératif, avec comparaison de deux stratégies de partitionnement. Le notebook personnel ([assignment3_esiee.ipynb](#)) reprend la même logique que le lab mais introduit une

différence technique importante : l'utilisation explicite de `new_ranks.checkpoint()` à chaque itération pour briser le lineage.

Recours à l'IA

Compréhension et résolution de l'erreur `Py4JJavaError` : L'erreur rencontrée dans le Lab 3 (`Cell In[5], line 38 – new_ranks.count()`) était due à un lineage trop long sans checkpoint. J'ai soumis la stack trace à Claude, qui a identifié immédiatement que le problème provenait d'un lineage excessif dans une boucle itérative. Il m'a recommandé d'appeler `spark.sparkContext.setCheckpointDir("checkpoint_tmp")` en début de session (déjà fait) et d'insérer `new_ranks = new_ranks.checkpoint()` avant `new_ranks.count()` à chaque itération. Cette correction a été appliquée dans l'assignment et a résolu l'erreur.

Différence entre `cache()` et `checkpoint()` dans un contexte itératif : Suite à la correction précédente, j'ai demandé à Claude d'expliquer plus précisément pourquoi `cache()` seul ne suffisait pas. Il m'a expliqué que dans Spark, le lineage (le graphe d'opérations permettant de reconstruire un `DataFrame`) s'accumule à chaque itération : à l'itération 5, `new_ranks` encode implicitement 5 niveaux de jointures et d'agréations. `cache()` mémorise le résultat mais conserve ce lineage en mémoire, ce qui finit par causer un `StackOverflowError` dans le driver Java. `checkpoint()` sérialise le `DataFrame` sur disque et repart d'un lineage vide, ce qui permet aux itérations de s'exécuter en temps et mémoire constants.

Signification des métriques de shuffle dans `lab3_metrics_log.csv` : Le notebook génère un CSV avec les colonnes `shuffle_read_bytes_ui` et `shuffle_write_bytes_ui` à remplir manuellement depuis la Spark UI. J'ai demandé à Claude comment localiser ces métriques précisément dans l'interface. Il m'a indiqué qu'il fallait aller dans l'onglet Stages, cliquer sur un stage correspondant à une itération PageRank, et lire les métriques dans la section Summary Metrics de la page de détail du stage. Pour les itérations du Run 1, le shuffle est visible dans le stage de `groupBy("id")` qui agrège les contributions.

Interprétation du speedup de 1.36x : Le Run 1 totalisait 3 720 ms et le Run 2 2 737 ms, soit un speedup de 1.36x visible dans la capture d'écran de l'assignment. J'ai demandé à Claude si ce speedup était principalement dû au repartitionnement ou au checkpoint. Il m'a répondu que l'usage du checkpoint dans les deux runs nivelle en partie les avantages du repartitionnement (car l'I/O disque du checkpoint introduit un bruit constant), mais que le repartitionnement reste bénéfique en réduisant la quantité de shuffle nécessaire lors des jointures. Sur un graphe plus grand, l'impact du repartitionnement serait proportionnellement plus marqué que le coût fixe du checkpoint.

Construction du graphe et résultats

Le graphe est construit à partir du fichier `match_picks.csv` (12 478 octets, 1 000 picks pour 200 matchs de 5 héros chacun). L'auto-jointure produit 586 arêtes uniques et 50 sommets. Rendu bidirectionnel, le graphe contient donc 1 172 arêtes orientées.

La skew analysis montre que hero_01 a le degré le plus élevé (49), hero_02 (48), et que la moyenne est de 23.44 avec un écart-type de 11.02. Le ratio max/moyenne de $49/23.44 \approx 2.09$ confirme une asymétrie modérée justifiant l'expérimentation du repartitionnement.

Run 1 — Partitionnement par défaut avec checkpoint

Les 10 itérations du Run 1 convergent de `max_delta=0.028153` (itération 0) à `max_delta=0.000000` (itération 9). Le temps total est de 3 720 ms sur 10 itérations, soit une moyenne de 372 ms par itération. Les temps descendent progressivement de 716 ms (itération 0, la plus coûteuse en raison de l'initialisation des structures) à 266 ms (itération 9). Les rangs finaux placent hero_01 en tête (0.0397), suivi de hero_02 (0.0388) et hero_03 (0.0364), cohérent avec leur degré plus élevé dans le graphe.

Les `shuffle_read_bytes_ui` et `shuffle_write_bytes_ui` renseignés manuellement depuis la Spark UI sont de 472 octets pour toutes les lignes des deux runs dans l'assignment (valeur obtenue depuis les stages de l'UI), contre 11 824 octets en lab et 19 318 en lab Run 2. La différence s'explique par l'utilisation du checkpoint dans l'assignment : le checkpoint matérialise les données sur disque avant chaque jointure, ce qui réduit le shuffle visible dans les stages Spark car les données sont relues directement depuis le fichier checkpoint plutôt que recalculées et redistribuées.

Run 2 — Repartitionnement par clé `src` avec checkpoint

Le Run 2 repartitionne les arêtes et les rangs en 8 partitions par la clé `src / id`. Le temps total est de 2 737 ms, soit un speedup de 1.36x par rapport au Run 1. Les rangs finaux sont identiques au Run 1 (correctness validée), et la convergence suit exactement la même trajectoire de `max_delta`, ce qui est attendu car le repartitionnement ne change pas le résultat mathématique de PageRank, seulement son efficacité d'exécution.

Analyse comparative

Le speedup de 1.36x en assignment est meilleur que le 1.05x observé en lab. La différence s'explique par l'usage du checkpoint : en lab, les itérations sans checkpoint étaient progressivement ralenties par l'accumulation du lineage, ce qui pénalisait davantage le Run 1 (sans repartitionnement) car il avait un lineage plus complexe à gérer. En assignment, les deux runs partent d'un lineage frais à chaque itération grâce au checkpoint, et c'est donc purement l'effet du repartitionnement qui est mesuré, donnant un speedup plus propre et plus élevé.